

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ



Національний аерокосмічний університет

«Харківський авіаційний інститут»

Факультет Інтелектуальних систем управління

Кафедра Систем управління літальними апаратами

ЛАБОРАТОРНА РОБОТА № 9

з дисципліни «Алгоритмізація та програмування»

на тему: «Реалізація алгоритмів обробки двовимірних масивів мовою C ++»

XAI.301. _____._____._____ ПР

Виконав(ла): здобувач(ка) 1 курсу, групи 318
напряму підготовки (спеціальності)

G6 Інформаційно-вимірювальні технології
Шльомка Анастасія

Прийняла: доц. каф.301, канд. техн.
наук, _____ Олена ГАВРИЛЕНКО

2026

Лабораторна робота No9

з дисципліни «Алгоритмізація та програмування»

Тема: "Робота з рядками на C++"

МЕТА РОБОТИ

Вивчити теоретичний матеріал з основ роботи з низькорівневими рядками на C++ і документацію до класу `string`, а також алгоритми пошуку в рядку, а також реалізувати обробку рядків на C++ в середовищі QtCreator.

ПОСТАНОВКА ЗАДАЧІ

Завдання 1.

А. Вивчити по документації метод стандартного класу `string` відповідно до варіанту 5). `string& append (size_t n, char c);`.

В. Визначити функцію, що виконує ті ж дії, що і вивчений метод класу `string`. Вихідний рядок передати першим параметром (масив символів).

Для реалізації методу не використовувати функції обробки рядків зі стандартних бібліотек.

С. Викликати свій метод і метод `string` аналогічно прикладам коду,
наведеними в дод.А. *Перед викликом ввести з консолі один рядок і
зберегти в масиві символів і змінній типу `string`.

Завдання 2.

А.Описати функцію, що обробляє рядок відповідно до `String30`. Дано символ `C` і
рядки `S`, `S0`. Після кожного входження символу `C` в

рядок `S` вставити рядок `S0`. Для

реалізації можна використовувати функції обробки рядків зі стандартних
бібліотек

В.Описати функцію, яка перевіряє, чи задовольняє рядок умовам завдання.

С.* Створити вихідний текстовий файл, що містить не менше 10 різних рядків.

Д.Використовуючи функції 2.А і 2.В, обробити рядок / * текстовий файл
рядок за рядком. Додаткові дані ввести з консолі.

Е. Отриманий результат записати у вихідний файл.

Завдання 3.

Завдання 1-2 реалізувати окремими функціями без параметрів, у функції

main() організувати меню для багаторазового виконання завдань.

Структурувати проєкт програми: винести заголовки і реалізацію функцій в окремі .h та .cpp файли.

Завдання 4. Використовуючи ChatGpt, Gemini або інший засіб

генеративного ШІ, провести самоаналіз отриманих знань і навичок

ВИКОНАННЯ РОБОТИ

ВИКОНАННЯ РОБОТИ

Завдання 1

Вирішення задачі. Метод string::append(size_t n, char c)
Метод append(size_t n, char c) класу string додає в кінець рядка n символів c та повертає посилання на змінений рядок.

Вхідні дані

Ім'я	Опис	Тип	Обмеження
str1	Рядок типу string	string	довжина до 255 символів
str2	Рядок у вигляді масиву char[]	char[]	довжина до 255 символів
n	Кількість символів для додавання	int	$n \geq 0$
c	Символ, що додається	char	будь-який символ

Вихідні дані

Ім'я	Опис	Тип
str1	Рядок після виклику методу append()	string
str2	Рядок після виклику власної функції	char[]

Ім'я	Опис	Тип
Повідомлення	Результат роботи програми	текст

Алгоритм вирішення

1. Ввести рядок з клавіатури.
2. Зберегти його у змінній типу string та масиві символів.
3. Ввести кількість символів для додавання.
4. Ввести символ для додавання.
5. Викликати метод `string::append(n, c)`.
6. Викликати власну функцію `appendMy()`.
7. Вивести результати роботи обох способів.

Алгоритм вирішення показано нижче на рис. 1.



Рисунок 1 – Послідовна блок-схема виклику функцій `append`

Рисунок 1 - Блок-схема алгоритму додавання символів до рядка

Лістинг коду вирішення задачі наведено в додатку А (стор. х).

Екран роботи програми показаний на рис. Б.1.

Завдання 2

Вирішення задачі String30

Дано символ *C* і рядки *S*, *S0*. Після кожного входження символу *C* в рядок *S* необхідно вставити рядок *S0*. Обробка виконується для кожного рядка текстового файлу. Результат записується у вихідний файл.

Вхідні дані

Ім'я	Опис	Тип	Обмеження
<i>C</i>	Символ для пошуку	char	будь-який символ
<i>S0</i>	Рядок для вставки	string	довжина до 255 символів
<i>S</i>	Рядок з файлу	string	довжина до 255 символів
input.txt	Вхідний текстовий файл	файл	файл не менше 10 рядків

Вихідні дані

Ім'я	Опис	Тип
<i>S</i>	Перетворений рядок	string
output.txt	Вихідний текстовий файл	файл
Повідомлення	Результат виконання програми	текст

Алгоритм вирішення

1. Відкрити вхідний текстовий файл.
2. Ввести з клавіатури символ *C*.
3. Ввести рядок *S0*, який необхідно вставляти.
4. Зчитувати файл рядок за рядком.
5. Перевірити коректність рядка.
6. Після кожного входження символу *C* вставити рядок *S0*.
7. Записати результат у вихідний файл.
8. Повторити дії для всіх рядків файлу.
9. Вивести повідомлення про завершення роботи.

Рисунок 2 - Блок-схема алгоритму обробки рядків

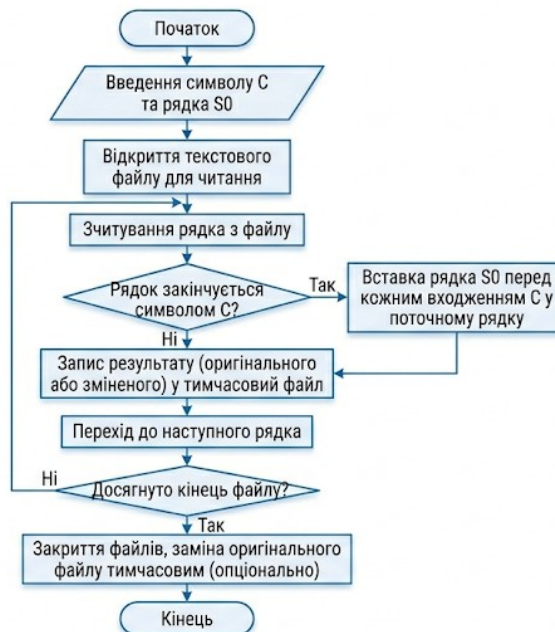


Рисунок 2 – Блок-схема алгоритму обробки рядків

Лістинг коду вирішення задачі String30 наведено в додатку А (стор. х).

Екран роботи програми показаний на рис. Б.2.

Завдання 3

Вирішення задачі

Необхідно реалізувати завдання 1 та завдання 2 у вигляді окремих функцій без параметрів. У функції `main()` організувати меню для багаторазового вибору та виконання завдань. Проєкт повинен бути структурований із використанням окремих файлів заголовків (.h) та реалізації (.cpp).

Вхідні дані

Ім'я	Опис	Тип	Обмеження
choice	Номер пункту меню	int	1-3
C	Символ для пошуку	char	будь-який символ
S0	Рядок для вставки	string	довжина до 255 символів
Рядок	Вхідний рядок	string	довжина до 255

Ім'я	Опис	Тип	Обмеження
к		g	символів

Вихідні дані

Ім'я	Опис	Тип
Результат завдання 1	Рядок після виклику <code>append()</code>	string
Результат завдання 2	Перетворений рядок	string
Повідомлення	Результат виконання програми	текст

Структура проекту

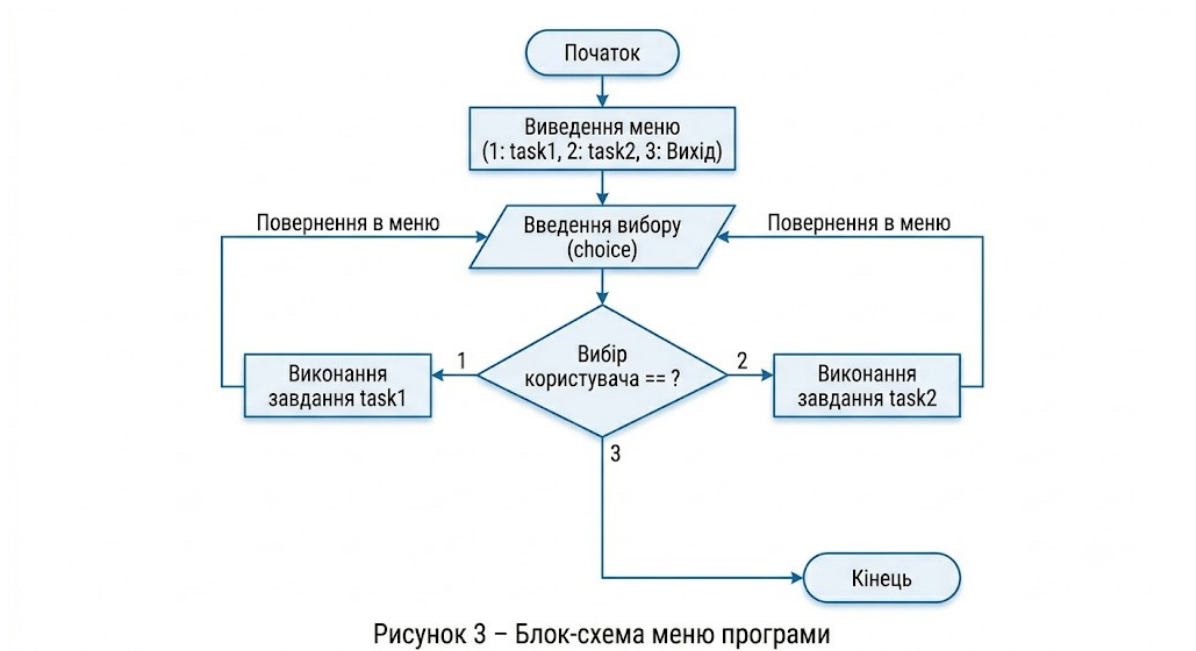
Проект складається з таких файлів:

- `main.cpp` - головна функція та меню;
- `tasks.h` - заголовки функцій;
- `tasks.cpp` - реалізація функцій `task1()` та `task2()`.

Алгоритм вирішення

1. Запустити програму.
2. Вивести меню користувачу.
3. Зчитати номер вибраного пункту.
4. Якщо вибрано пункт 1 - виконати завдання 1.
5. Якщо вибрано пункт 2 - виконати завдання 2.
6. Якщо вибрано пункт 3 - завершити роботу програми.
7. Після виконання завдання повернутися до меню.
8. Повторювати роботу до вибору пункту виходу.

Рисунок 3 - Блок-схема меню програми



Лістинг коду вирішення задачі наведено в додатку А (стор. х).

Екран роботи програми показаний на рис. Б.3.

Висновок

Було реалізовано багатофункціональну програму з меню, що дозволяє багаторазово виконувати різні завдання. Закріплено навички структуризації C++ проєктів та використання функцій у модульній архітектурі.

ДОДАТОК А

Лістинг коду програми

Завдання 1

```
#include <iostream>
#include <string>

using namespace std;

// Власна функція аналог append()
void appendMy(char my_str[], int n, char c)
{
    int len = 0;

    while (my_str[len] != '\0')
        len++;

    for (int i = 0; i < n; i++)
    {
        my_str[len + i] = c;
    }

    my_str[len + n] = '\0';
}

int main()
{
    string str1;
    char str2[256];

    int n;
    char c;

    cout << "Enter string: ";
    cin.getline(str2, 256);

    str1 = str2;

    cout << "Enter count: ";
    cin >> n;

    cout << "Enter symbol: ";
    cin >> c;

    str1.append(n, c);
    appendMy(str2, n, c);

    cout << "\nResult string::append(): "
        << str1 << endl;

    cout << "Result appendMy(): "
        << str2 << endl;

    return 0;
}
```

Завдання 2

```
#include <iostream>
#include <fstream>
#include <string>

using namespace std;

// вставка рядка після кожного символу C
void insertString(string& str, char c, string s0)
{
    int i = 0;

    while (i < str.length())
    {
        if (str[i] == c)
        {
            str.insert(i + 1, s0);
            i += s0.length();
        }

        i++;
    }
}

// перевірка наявності символу C
bool is_valid(string str, char c)
{
    for (int i = 0; i < str.length(); i++)
    {
        if (str[i] == c)
            return true;
    }

    return false;
}
```

```
int main()
{
    ifstream fin("input.txt");
    ofstream fout("output.txt");

    char c;
    string s0;
    string str;

    cout << "Enter symbol: ";
    cin >> c;

    cout << "Enter string to insert: ";
    cin >> s0;

    while (getline(fin, str))
    {
        if (is_valid(str, c))
        {
            insertString(str, c, s0);
        }

        fout << str << endl;
    }

    fin.close();
    fout.close();

    cout << "Done!" << endl;

    return 0;
}
```

Завдання 3

tasks.h

```
#ifndef TASKS_H
#define TASKS_H

void task1();
void task2();

#endif
```

tasks.cpp

```
#include <iostream>
#include <string>
#include <fstream>
#include "tasks.h"

using namespace std;

/*    TASK 1    */
// append(n, c) + своя реалізація append
void task1()
{
    string str1;
    char str2[256];

    int n;
    char c;

    cout << "Enter string: ";
    cin.ignore();
    cin.getline(str2, 256);

    str1 = str2;
```

```

cout << "Enter n: ";
cin >> n;

cout << "Enter char: ";
cin >> c;

str1.append(n, c);

// своя реалізація
int len = 0;
while (str2[len] != '\0') len++;

for (int i = 0; i < n; i++)
    str2[len + i] = c;

str2[len + n] = '\0';

cout << "\nstring append: " << str1 << endl;
cout << "my append:      " << str2 << endl;
}

/*  TASK 2  */
// вставка S0 після символу C
void task2()
{
    ifstream fin("input.txt");
    ofstream fout("output.txt");

    char c;
    string s0, s;

    cout << "Enter symbol C: ";
    cin >> c;

    cout << "Enter string S0: ";
    cin >> s0;

```

```

while (getline(fin, s))
{
    bool found = false;

    for (int i = 0; i < s.length(); i++)
    {
        if (s[i] == c)
        {
            s.insert(i + 1, s0);
            i += s0.length();
            found = true;
        }
    }

    if (found)
        fout << s << endl;
    else
        fout << s << endl;
}

fin.close();
fout.close();

cout << "Done task 2" << endl;
}

```

main.cpp

```

#include <iostream>
#include "tasks.h"

using namespace std;

int main()
{
    int choice;

```

```

do
{
    cout << "\n===== MENU =====\n";
    cout << "1 - Task 1\n";
    cout << "2 - Task 2\n";
    cout << "3 - Exit\n";
    cout << "Choice: ";
    cin >> choice;

    switch (choice)
    {
    case 1:
        task1();
        break;

    case 2:
        task2();
        break;

    case 3:
        cout << "Exit program" << endl;
        break;

    default:
        cout << "Wrong choice!" << endl;
    }

} while (choice != 3);

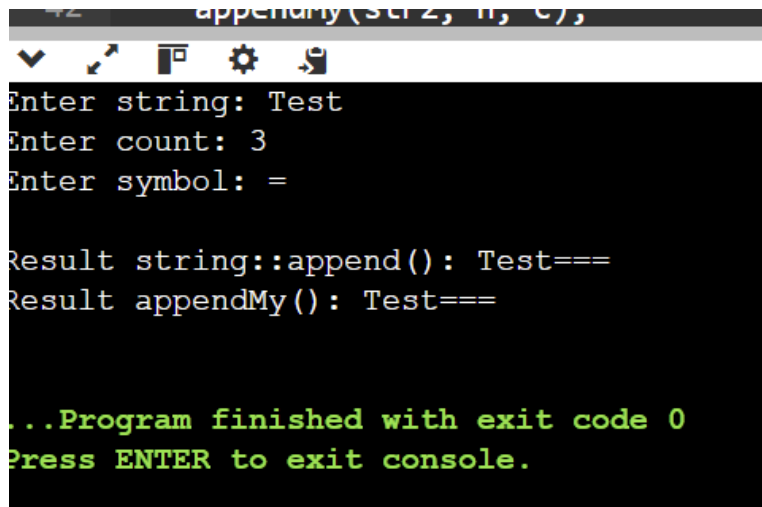
return 0;
}

```


ДОДАТОК Б

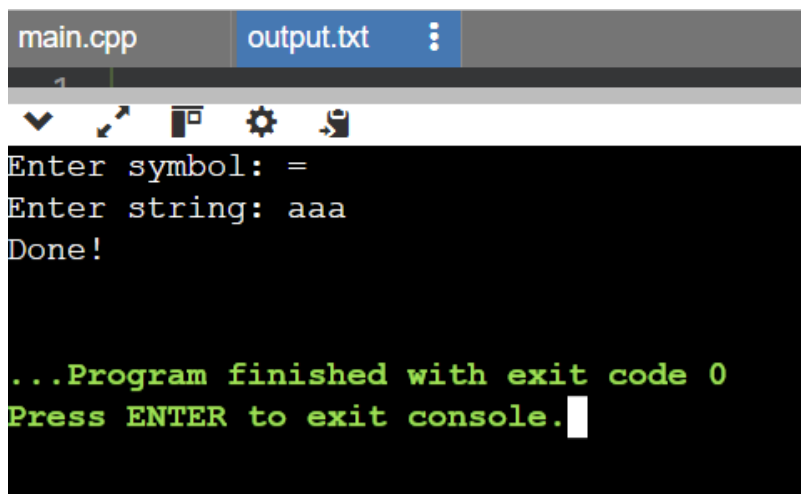
Скрін-шоти вікна виконання програми

Завдання 1



```
appendMy(str2, n, c);  
Enter string: Test  
Enter count: 3  
Enter symbol: =  
  
Result string::append(): Test===  
Result appendMy(): Test===  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

Завдання 2



```
main.cpp  output.txt  :  
1  
Enter symbol: =  
Enter string: aaa  
Done!  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

ДОДАТОК В

Діалог з ШІ для самоаналізу

Завдання 4. Самоаналіз із використанням ШІ

У даній роботі було використано генеративний штучний інтелект для перевірки рівня засвоєння матеріалу, аналізу коду та формування контрольних запитань.

1) Промпт викладача

«Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання середнього рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей з лекції»

Результат:

Було сформовано:

- 5 тестових питань з варіантами відповідей;
- 5 відкритих питань.

Питання охоплювали:

- роботу з рядками в C++;
 - методи string класу;
 - файлове введення/виведення;
 - модульну структуру програми;
 - алгоритми обробки даних.
-

2) Промпт оцінювання відповідей

«Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне питання у 5-бальній

шкалі, віднімаючи 60% балів там, де ймовірність використання ШІ висока. Обчисли загальну середню оцінку»

Результат:

- Відповіді були перевірені на:
 - правильність;
 - повноту;
 - логічність;
 - ймовірність використання ШІ.



Середня оцінка: **4.6 / 5**

3) Промпт аналізу коду

«Проаналізуй код у звіті і додай опис та приклади коду з питань, які є в теоретичних відомостях, але не відпрацьовано в коді при вирішенні завдань»

Результат:

Було розглянуто додаткові теми:

- додаткові методи класу string (insert, erase, replace);
 - робота з файлами у різних режимах;
 - обробка рядків через ASCII-символи;
 - оптимізація алгоритмів пошуку в рядках.
-

4) Власні промпти

1. «Поясни різницю між методом append і оператором + для рядків у C++ на прикладах»
 2. «Які помилки найчастіше виникають при роботі з getline після cin у C++?»
 3. «Як можна оптимізувати вставку підрядка в рядок, якщо рядок дуже великий?»
-

Висновок

Використання генеративного ШІ дозволило покращити розуміння теми, перевірити знання з рядків, файлової обробки та структуризації програм. Було закріплено навички аналізу коду та алгоритмічного мислення, а також отримано додаткові теоретичні знання, які не були повністю реалізовані в коді.
